WARWICK
THE UNIVERSITY OF WARWICK

Manuscript version: Author's Accepted Manuscript
The version presented in WRAP is the author's accepted manuscript and may differ from the published version or Version of Record.

Persistent WRAP URL:
http://wrap.warwick.ac.uk/177110

How to cite:
Please refer to published version for the most recent bibliographic citation information. If a published version is known of, the repository item page linked to above, will contain details on accessing it.

Copyright and reuse:
The Warwick Research Archive Portal (WRAP) makes this work by researchers of the University of Warwick available open access under the following conditions.

Licensed under the Creative Commons Attribution 2.0 Generic (CC BY 2.0)
https://creativecommons.org/licenses/by/2.0/



Publisher's statement:
Please refer to the repository item page, publisher's statement section, for further information.

For more information, please contact the WRAP Team at: wrap@warwick.ac.uk.

warwick.ac.uk/lib-publications

Vicarious Coding:
Breaching Computational Opacity in the Digital Era

Marjolaine Rostain
Warwick Business School
Scarman Road
Coventry CV4 7AL, United Kingdom
Marjolaine.Rostain@wbs.ac.uk

Ruthanne Huising
Emlyon Business School
23 avenue Guy de Collongue
Ecully, France
huising@em-lyon.com

Acknowledgements. This work benefited from the generous feedback of the following people: Diane Bailey, Beth Bechky, Daisy Chung, Jean Clarke, Lisa Cohen, Kasper Trolle Elmholdt, Anne-Laure Fayard, Maximilian Heimstädt, Mayur Joshi, Arvind Karunakaran, Tomi Koljonen, Elina Mäkinen, Anca Metiu, Pedro Monteiro, Nicholas Occhiuto, Samantha Ortiz, Pauli Pakarinen, Nevena Radoynovska, Kathrin Sele, Anastasia Sergeeva, and Jeroen Struben. We also thank Andrew Nelson and three anonymous reviewers.

VICARIOUS CODING:
BREACHING COMPUTATIONAL OPACITY IN THE DIGITAL ERA

ABSTRACT

Digital representations are ubiquitous in the workplace. Screen displays, forecasts, simulations, indicators, multi-dimensional models, figures, and images are increasingly central to work of all kinds. Representations are simultaneously transparent and opaque. They contain and reveal information about the organization. At the same time, they conceal the computational work used to convert data about the physical world into abstract depictions. Computational opacity is consequential when representations become misaligned with the physical world they depict. We examine how computational opacity can be breached, allowing non-programmers to repair misalignments between representations and the physical world. Drawing on an ethnography of a machine-shop floor, we show how operators develop *practical computational literacy skills*—the capacity to visualize and talk about physical objects and processes independent of them; to translate this noncomputational thinking and talking into computational symbols, syntax, structure, and assumptions; and to create computational solutions. We show how operators develop this skill vicariously, observing programmers as they solve problems. We contribute to understanding how, in an increasingly digitized workplace, those without programming capacities may decrease their dependence on programmers and increase their capacity to create and alter representations of the physical world.

INTRODUCTION

Digital representations are ubiquitous in the workplace. Screen displays, forecasts, simulations, indicators, multi-dimensional digital models, figures, and images are increasingly central to work of all kinds. Representations synthesize the physical world, facilitating the visualization and interpretation of complex phenomenon (e.g., Beunza & Stark, 2012). Graphs and images of physical systems communicate complicated empirical patterns in an accessible manner (Barley, 2015). Three-dimensional models show a physical object to be constructed in detail (Boland, Lyytinen & Yoo, 2007). Control screens portray a physical production process in real time (Zuboff, 1988). The creation and use of representations have become "vital organizational activities" (Kallinikos, 2009:184).

Representations are simultaneously transparent and opaque. They contain and reveal information that has the potential to increase the visibility of organizational operations, actors, and untapped opportunities. Representations of organizational operations generate images and

knowledge of activities that were previously out of view (Zuboff, 1988). They create shared understandings of work processes (Bechky, 2003a; Carlile, 2002) and organizational patterns and routines (Huising, 2019, Leonardi & Treem, 2020). They offer "visibility into others' tasks that can lead people to make connections between and combinations of ideas that result in new products, processes, or services" (Leonardi & Bailey, 2017: 117). The relatively frictionless reproduction and circulation of representations and the information they hold across time, space, and roles has the potential to alter the distribution of knowledge across the organization (Bailey et al., 2012: 1485 Leonardi & Bailey, 2008, 2017).

At the same time representations conceal the computational work used to create them. In the digital era, representations are the product of computational thinking and coding that convert data about the physical world into a more or less abstract depictions. Representations may be computationally opaque to their users in two ways. First, representations may be experienced as closed when users do not have access the computational logic and coding used to create them. Second, representations may be experienced as inflexible or rigid when users do not have the capacity to modify and update them. The computational opacity of representations stems from "technical illiteracy" which creates an "opacity stemming from the current state of affairs where writing (and reading) code is a specialized skill" (Burrell, 2016: 2). Writing and reading code is the primary skill of programmers, software engineers, and developers who work in specialized languages (e.g., Python, JavaScript, Java, C++), using formal logic, symbols, and syntax to transform the physical world into representations. For other workers, the computational thinking and coding embedded in representations —including assumptions, logic, commands, and errors— are black boxed.

Computational opacity is problematic when representations are observed to be misaligned with the physical world. Misalignments between the physical world and representations are observed across the literature. Shelves may be stocked with medication while a screen display indicates that shelves are bare (Barrett, Oborn, Orlikowski, & Yates, 2012). Satellites may be on course even though incoming predictions indicate a problem (Mazmanian et al., 2014). Results of a physical crash test may differ from those of a simulated crash test (Bailey et al., 2012). Although representations excel at precision, their capacity to capture the "real" is always incomplete: this is primarily because "there is more entropy in the real world than there is in your computer" (Turkle, 2008: 81).

Reliable agreement between representations and the physical world is crucial for guiding organizational action (Bailey et al., 2012). Agreement or realignment depends on the computational work of programmers, software engineers, and developers. They must reconsider their abstraction and conceptualization of the physical world and check and modify the coding. Workers who do not know how to read and write the code that informs representations that are central to their work – pharmacists (Barrett et al. 2012), pilots (Oliver et al. 2017), art historians (Sachs 2020) – are unable to trouble shoot and fix representations that deviate from the physical world. The organization's dependence on computational workers creates frustration, delays, and bottlenecks in daily operations as well as any efforts to adapt and innovate. The computational opacity of representations reveals an important digital divide in organizations between those who create representations and those who work from them.

In this paper, we examine how non-programmers breach the computational opacity of representations. Drawing on an ethnographic study of a computer numeric control (CNC) machine shop, we show how operators, who work from representations created by programmers,

accumulate a set of skills that build to what we refer to as practical computational literacy. *Practical computational literacy* is the capacity to visualize and talk about physical objects and processes independent of them; to translate this noncomputational thinking and talking into computational symbols, syntax, structure, and assumptions; and create computational solutions that realign representations and the physical world. Machine operators do not become programmers, but they learn a selective, practical set of skills needed to effectively diagnose and fix the errors independent of programmers. In doing so, they breach the computational opacity of the representations they work from and become less dependent on programmers to rethink and alter representations.

Beyond showing what the machine operators learn, we show the vicarious means through which this skill is learned. Practical computational literacy is learned neither through situated practice within a community of practice (with machine operators) nor through legitimate peripheral participation (with programmers) (Lave & Wenger, 1991). Instead, it occurs as machine operators observe programmers fix problems, collecting bits of talk, logic, and technique. The learning happens vicariously, through "a process of observing and imitating the actions" of others (Myers, 2022: 379). Shedding light on a means through which workers may breach computational opacity contributes to understanding how, in the face of an increasingly digitized workplace, workers without computational capacities may learn the computational work that informs the representations they work with and periodically, independently rethink and reprogram representations of their work.

WORKING WITH REPRESENTATIONS

Representations are signs that "have both a meaning and a referent: they enable interpretations among those who know how to read them (meaning) and also point to something definite in the world, a physical entity (referent)" (Bailey, Leonardi, & Barley, 2012: 1486).

Transformational technologies are used to produce representations, drawing on computational processes to convert information about the physical world into abstract depictions (Leonardi & Bailey, 2008). Architects, for example, create three-dimensional blueprints (representations) of buildings (physical world) using architectural software such as CATIA or Rhino (transformational technology) instead of hand-drawing a building plan (Boland, Lyytinen, & Yoo, 2007; see Table 1 for more examples). Creating representations requires abstracting away from the physical world, transforming from the concrete to the conceptual and then coding words to symbols.

Insert Table 1 about here

Although representations are ubiquitous in the workplace, the role of representations in work and the way in which occupational groups work with representations vary. We distinguish between three points on a continuum: those who work within representations; those who work closely and iteratively between representations and the physical world; and those who work from representations, manipulating physical objects and processes as guided (summarized in Table 2). First, work *within* representations involves combining abstract models and reams of data about the physical world, programming skills, and transformational technologies to create representations of the physical world (Leonardi & Bailey, 2008). Moving across abstract ideas of physical processes, the data these processes generate, and transformational technologies, these workers experience representations as computationally transparent because they have the skills to create and recreate representations. The representations are experienced as both pliable, meaning they can modify them on an ongoing basis, and as open, meaning they understand the models, data, and computational processes through which they are created. They can, for

example, alter theoretical assumptions or change parameters, thereby remaking representations and experimenting with multiple versions of representations.

Insert Table 2 about here

For those who work *within* representations, any misalignment between a representation and the physical world can be solved by reconceptualizing the representation and altering the computer code, thereby remaking representations. To do to this, automotive engineers, software engineers, epidemiologists, atmospheric scientists periodically touch base with the physical world they are representing (Bailey et al., 2012). In the case of an automobile manufacturing company, engineers became frustrated when the "simulated passenger dummy seemed to be hit by an expanding airbag with more force than had the real dummy in the physical test" (Bailey et al., 2012: 1496). Comparing both the simulation program and the physical crash, the engineers identified a problem with the dimensions of the physical passenger-assist bar that sits above the airbag (within the plastic encasing). This involved testing and developing a theoretical explanation for the misalignment and modifying the parameters (computational language) inside the simulation software (transformational technology) so that the new simulation reflected the actual car crash. The engineers identified and modified the erroneous parameters in the computational model to create a new representation aligned with the physical world. Those who work *within* experience representations as pliable and open, as they have the skills to modify and update the computer code and create new representations.

Second, those who work *between* representations and the physical world (e.g., Barley & Bechky, 1994; Orr, 1998) create linkages between these worlds "with one foot in the material world and the other in a world of representations" (Barley, 1996: 418). Technicians are often responsible for this work, transforming the physical world into representations via the user-

friendly GUI interfaces of transformational technologies. For example, radiology technicians produce radiological images (representation) of a patient's body (physical) using a computerized tomography (CT) scanner (transformational technology). They have operational knowledge of the technologies that produce representations (Black et al., 2004). Representations are "pliable" in the sense that the worker can continuously generate representations with the transformational technology (Kallinikos et al., 2010). However, those who work *between* representations and the physical world are not trained or expected to modify the computational language inside the transformational technology. The transformational technology is experienced as semi-transparent or as a "grey box" (Fujiumura, 1992), as these workers are able to use it to produce and reproduce representations but the computational mechanisms through which representations are created in the technology are beyond their reach. The representations produced are thus experienced as "closed." For example, radiology technicians are trained to operate the CT scanner to produce a radiological image but do not have knowledge about the underlying principles and rules of the CT system itself or skills to change the programming within it (Barley, 1986).

Those who work *between* representations and the physical world address misalignments by altering the representation to better reflect the physical world. They learn to work around the semi-transparency of the computational work and processes that produce representations. Workarounds emerge from situated learning within a community of practice (Lave & Wenger, 1991). For example, computer technicians use a peer-produced software-problem bulletin to supplement the inadequate formal instructions or representations of how to install a new software application (Pollock, 2005). Photocopy technicians narrate machine breakdowns—telling stories, narrating the problem, developing causal linkages, and generating possible diagnoses—that

differ from their manuals (Orr, 1996). Junior bankers learn reasoning practices from senior bankers that help identify and solve misalignment between automated recommendations and the particulars of a case by altering recommendations (Anthony, 2021). These workers reconstruct the representation based on within-community-of-practice reasoning and without altering the underlying computational processes that create the representation.

Third, work may be done *from* representations (Barrett et al., 2012; Bechky, 2003a; Swan, Bresnen, Newell, & Robertson, 2007). These workers are in direct contact with the physical world and are responsible for manipulating, producing, and orchestrating it as instructed by representations. Although they work intimately with the physical world, they are distant from the knowledge and technologies that create the representations that are inputs and guides to their interactions with the physical world. This includes shop-floor workers who work from representations (e.g., assemblers, not technicians, in Bechky, 2003a, 2003b) and members of professions who use representations to make decisions (e.g., human resource managers in van den Broek, et al. 2021). For these workers, representations are experienced as opaque, both inflexible and closed. They neither have the capacity "to modify and update [them] continuously and systematically" (Kallinikos, Aaltonen, & Marton, 2013: 358) nor can they access the underlying principles or rules of the program that create them (Jordan, 2009).

Those who work *from* representations deal with misalignment between the representation and the physical world by altering the physical world to align with representations. Workers may develop or appropriate tools that they integrate into their work process to align the physical with the representation. These workarounds emerge from situated learning within the community of practice (Lave & Wenger, 1991). For example, machine operators in a furniture factory developed thousands of "jigs"—materials used in the machine setup to override given

representations and create required bends in the wood (Juravich, 2017). Without these jigs, the furniture produced would not align with the designs. Similarly, Houston, Gabrys, and Pritchard (2019) observe workers building additional equipment to facilitate the operation and use of digital sensors. Acoustic engineers, noting that acoustic measurements represented in architectural software do not reflect actual measures, use other measurement tools and manual calculations to design acoustic solutions that meet project specifications (Merschbrock & Figueres-Munoz, 2015). Such examples show how workers creatively employ materials at hand to ensure that their work with the physical world meets the specifications and demands found in the representations. Alignment is achieved by altering work with the physical rather than altering the computational makeup of the representation.

Overall, those who work *between* and *from* representations often resort to workarounds when they experience misalignments. The workarounds created by those who work *between* (alter representation superficially) and *from* representations (alter the physical) share two important features. First, workers reconstruct the representation or the physical world without altering the underlying computational processes that create the representation. Although realignment occurs, the computational conceptualization and operationalization of the physical world is left unchanged. Thus, these efforts may provide temporary solutions, as they do not uncover the source of error or address the fundamental errors, shortcomings, or deviations that create misalignment between the representation and the physical. Second, workarounds are developed within a community of practice and are based on how practitioners in this community think about and conceptualize the physical object or process to be represented (Brown & Duguid, 1991: 46). The altered representation reflects how bankers, for example, reason about company financials given their prior and current knowledge of markets. The alterations made do not

reflect the financial models, assumptions, computations, and codes that produced the original recommendations. In developing workarounds, the practitioner draws on the knowledge, mindset, and tools used within her own community of practice. Thus, the diagnosis and solutions do not build the capacity to think about and abstract from the physical world like the computer programmers, engineers, or developers who create representations. Overall, the community of practice works around computational opacity by using their collective understanding and established ways of thinking and working to alter representations. This can be problematic for the organization, as the computation underlying the representation remains incorrect and will continue to generate representations that are not aligned with the physical, requiring continuous reliance on workarounds.

These workarounds neither create long-term, fundamental changes to the computational processes that create them nor develop knowledge about the computational mindsets and skills need to create the representations. Therefore, when workarounds are insufficient, those who work *between* and *from* representations rely on programmers and computer scientists to reconceptualize and reprogram the representations. For example, accounting managers depend on programmers to modify input fields in their software (Boudreau & Robey, 2005). Human resource managers depend on computer scientists to make changes to the algorithm when the chosen candidates do not match their manual selections (van den Broek, Sergeeva, & Huysman, 2021). Pharmacist assistants depend on software engineers to solve an error related to barcodes (Barrett et al., 2012). This can be problematic for communities of practice that depend on programmers, developers, and engineers to reconceptualize and reprogram representations as it creates bottlenecks and dependencies at the point of production, reinforcing the use of workarounds rather than generating more fundamental computational capacities.

Learning Barriers for Newcomers and Outsiders

The possibilities or potential for people who are not formally trained or expected to have the capacity to design, write, or read code to learn computational thinking and skills is difficult. One of the few studies of outsider learning describes how aspiring coders, without computer science and engineering degrees, learn to code through intensive problem-solving assignments with peers and near peers (Kaynak, 2023). Eventually, through further internships and moving across roles, many do gain entrance into the occupation. Typically, entry into occupations such as software engineers, developers, and programmers requires formal training, situated learning opportunities such as internships, and then socialization into the occupation (Abbott, 1988; Bailey & Barley, 2011). Even where workers are members of an occupational community in which particular skills and abilities are expected, we have ample evidence that efforts of junior members of the community to learn are thwarted because they are denied full access to situated learning opportunities (Bharatan, Swan, & Oborn, 2022). It is through exceptional practices that novice members overcome temporal (O'Mahony & Bechky, 2006), spatial (Beane, 2019), and hierarchical (Kellogg, Meyer et al., 2021) learning barriers. By changing the "pattern of learning episodes and [the] distribution of actors across those episodes," novices may gain access to experiences that expand their knowledge and skills (Bailey & Barley, 2011: p. 266).

For outsiders—those without the formal education to be considered a member of an occupational community—the barriers to learning are even higher because they do not have opportunities for situated learning and socialization into the appropriate ways of thinking and acting. Situated learning within a community of practice occurs as senior practitioners allow novices to observe their work (Lave & Wenger, 1991). As novices observe and assimilate the practices and approaches of senior practitioners, they are given opportunities to participate in the work, at first peripherally, and over time, more centrally in the work of the community. This

move from legitimate peripheral practice to independent practice is gradual and guided, ensuring that community members learn the skills, dispositions, and conduct expected of members (Lave & Wenger, 1991; Van Maanen & Barley, 1984). In the absence of membership in an occupational community and admittance to a community of practice, the barriers to learning the knowledge and skills involved are high.

In this paper, we examine how workers, without formal training or legitimate peripheral participation, build computational skill. We find that computational skills are learned vicariously (Bandura, 1977; Manz and Sims, 1981). Through watching programmers solve problems, workers collect observations, information, and techniques that they cobble together to problem-solve independently. Learning occurs by observing and then imitating the moves of the programmers (Myers, 2018) and allows the workers we studied to apply these moves in response to other problems (Gioia and Manz, 1985). Programmers model or teach their work unintentional as the problem solve. As programmers solve problems they narrate their thought process, they translate between natural language and code, and they describe the changes they make to the programming script. Through this observation, workers develop the means to think abstractly about their work—as programmers do—and develop the ability to understand computational symbols, syntax, and structure in noncomputational terms and then use this understanding to propose computational solutions. This vicarious learning process appears similar in many respects to those observed among negotiators (Nadler, Thompson and van Boven, 2003), flight nurses (Myers, 2022), and publishers (Simon and Liberman, 2010).

RESEARCH SETTING

Machining and Machine Operators

This study focuses on the work of people who operate a three or five-axis computer numerical control (CNC) machine. These machines remove material from a block of aluminum

or other metal to create a desired shape. Industrial sociologists, historians, and labor economists have produced a vast corpus on machining work and technological change (Dean, Yoon, & Susman, 1992; Hazlehurst, Bradbury, & Corlett, 1969; Jones, 1982; Keefe, 1991; Noble, 1978; Shaiken, 1984). Seventy years ago, these machines were run by skilled craftsmen who worked from an engineer's drawing of the desired outcome or a sample they were expected to replicate, producing complex, small-batch products. The machines were automated over time using magnetic tape, punch cards, and eventually programming (Noble, 1978), with large-scale diffusion of computerized numeric control machines in the 1980s.

Machine Operators at Formco

The machine operators we studied worked at Formco,[1] a firm of 70 employees with revenues exceeding €7 million in 2018. Formco ranked 27th of 440 companies, based on 2018 revenue, in the French mold industry. It is divided into two business units located five kilometers apart in the same city. The firm was created in 1934 and is owned by two directors, each of whom manages a business unit. Formco produces molds in aluminum, resin, or wood for several industries, including packaged goods, automotive, and sports. The molds are used by Formco's clients to produce plastic goods.

A standard, formal division of labor is used to design and produce molds at Formco. Designers, programmers, operators, and fitters work in sequence according to highly standardized procedures outlining well-defined production steps. A production manager oversees the entire process, and each group of workers is managed by a supervisor. The key differences in the workers' responsibilities, tasks, and tools are summarized in Figure 1.

Insert Figure 1 about here

[1] The names of the organization and of the employees are pseudonyms.

Because our paper focuses on programmers and machine operators, we describe in detail their work only. The programmers work in a modern open office space, shared with designers, away from the noise and grime of the shop floor. Using computerized advanced manufacturing (CAM) software, they choose the tools, motion, speed, and path that the machine will use to transform the aluminum block into a mold. The CAM software has a programming interface and a machining simulation. A mold can be machined in several ways, and each programmer works the program differently—combining various choices in customized ways—to achieve a result. They choose the different programming steps from a dropdown menu in the programming interface, and the software transforms these choices into a programming script. When the programming is complete, they verify the work by running a machining simulation. Watching the simulation on their computer screen allows them to identify and correct any problems with the programming, such as collisions between the tool holder and the material, a rough finishing surface, or a drill going too deeply into the material. Following this, a tool sheet is generated automatically. The tool sheet, composed of two sections, will be given to the machine operators. The first section is a basic diagram of the raw materials and indicates how it should be positioned within the machine and where the machining will begin (origin point). The second is a table indicating the number of the program (the programming script related to the mold) to load on the machine and the summary of the steps the machine will go through to machine the product, indicating the tools the operators should insert and their positioning.

The machine operators work in a large shop without windows. The space is noisy with the screeching sounds of metal being cut and shaved, and the floor is dirty with scraps and lubricant. Machine operators wear coveralls and safety boots and are supplied with earplugs and gloves that they rarely use, as this gear interferes with the sensory nature of their work. They

need to hear the machines working and touch the molds to feel whether they are smooth. They wear their gloves when moving a finished mold to the fitters' workshop because the mold has sharp edges.

Each operator is assigned either to a three-axis machine or to a larger, five-axis machine placed approximately three meters apart. The machine operator begins each job by examining the designer drawing and tool sheet. He[2] then goes to the central computer on the shop floor, where the programs for all molds are stored. He selects the program number indicated on the tool sheet, the number of his CNC machine, and presses the upload button to transfer the program to the CNC machine. The operator then returns to his machine and prepares the necessary tools, aligns the right and the left of the material on a virtual axis, clamps down the material, and places the machine in the starting position. On average these steps take 45 minutes to one hour. Preparing the tools means selecting each cutting tool indicated on the tool sheet and verifying that it is not broken and has the right shape. The operator fixes the tool on the tool holder in what is called "the shop"—the sole part of the machine where operators can handle all the tools. He closes the door, selects the program number indicated on the tool sheet, and presses *start.* The machine creates the mold automatically. During the machining, the operator watches the machine. When it stops, he either changes the mold position for a new program or removes the finished mold. When the mold is finished, he verifies its dimensions and stamps his name on the plan, signifying that it is complete. He then takes the mold across the plant to the fitter area, at the opposite part of the plant.

RESEARCH METHODS

[2] Throughout the paper we refer to the operators and programmers as he/his/him, as all but one identify as men.

Data Collection

The first author conducted an eight-month ethnography at Formco from February 2018 to December 2018. Before beginning fieldwork, she reviewed extensive documentation of the firm provided by the CEO to understand the history of the firm, its organizational structure and governance, the client base, the technology used, and products produced. Preliminary fieldwork included one week of observation in each business unit, observing the entire workflow, from the design office to the fitting area on the shop floor. Twenty-eight interviews were conducted to understand the operations of the firm with nine machine operators, seven managers, four top-level managers (CEO, and previous and current business unit managers), two customers, two salespeople, three assistants, and the financial director. The interviews lasted between 45 minutes and three hours. They were recorded and transcribed. Upon reading and rereading data from this preliminary fieldwork, the first author became interested in the contrast between the specialization of the molds produced and the standardized, automated production process used.

Given this, the first author began conducting in-depth observations of the end-to-end work process, from the sales work to the finished project. She spent eight months, for two to four days per week, conducting ethnographic observation. She moved across the sales, design, programming, machining, and fitting departments, learning the work of each department and how this work is coordinated across them. This was done by shadowing workers in each department and spending time in their space as they worked, watching, and listening to what was happening. There was also time for informal conversation during the workday, and during coffee breaks, and lunch. Most of the time was spent on the shop floor, observing and interacting with machine operators. Data collection involved writing detailed field notes, collecting all documentation, taking photographs of the offices and workstations, and recording videos of the work. Field notes were typed up as soon as possible after leaving the field, usually each night.

Data Analysis

We began the analysis process by reading and rereading the data in an open manner, without a specific research question. At Formco, work is organized and conducted in a divided, sequential way. The role of the machine operator is to set up production, start the preprogrammed instructions, tend the machine while it operates, and pass the completed part to the fitter who completes the mold. The operators consider these activities to be button-pushing work. Supervisors treat operators as button pushers, monitoring and disciplining machine operators. This is a deliberate approach, as the CEO explained: "If one worker leaves, we have to replace him immediately with another one. We need to ensure that each worker is substitutable." However, we noticed that the machining of some molds broke this mechanical shop-floor routine. At these moments, the operators came alive, interacting with their co-workers and working with their machines in different ways.

We focused on understanding what triggered this change from solitary button-pushing to a range of interactions with others and different interactions with their machines. We examined the data to find the molds for which the first author observed production from start to finish. The first author reviewed the production of each of these 90 molds to identify those that proceeded in a button-pushing fashion [57 molds] and those that deviated from this routine [33 molds]. For these 33 molds we found that 9 were execution errors which involved the supervisor working with the machine operator to correct the problem. These were generally recently hires, and the interactions were simple. For the other 24 molds, the errors were more complex and involved more extended interactions with programmers and the machines. We compiled all the data related to these 24 machining jobs. The first author read and reread these data and coded them, paying attention to the source of the problem, the actors involved, and the actions involved in

diagnosing and fixing the problem. As she was coding, she compared and contrasted the 24 machining jobs to understand how they were the same and how they were different.

Based on the coding, we first tried to understand the various sources of the problems, identifying two main sources—programming errors and deviations in the material or machine. As we analyzed these errors, we noted that they were beyond the control of machine operators and required that their work be reconceptualized and reprogrammed. We then focused on understanding the process through which reconceptualization and reprogramming occurred. In 6 cases the machine operators fixed the problem by independently reconceptualizing and reprogramming their work. In 5 cases the machine operators diagnosed the problem independently but relied on the programmer to develop the solution or program the solution. In 13 cases the machine operator observed while the programmer fixed the problem. We then organized the machining jobs chronologically, observing machine operators seemed to be learning something from the programmers that they then used later to independently diagnose and solve problems.

Our analysis then shifted to examining what was being learned and how. The first author recoded the 13 cases in which the machine operators watched the programmers work. At this stage of analysis, we noted the role of the tool sheet, the programming software, and the machine in the diagnosis and solving of the problem. We developed, based on this coding, a preliminary model of three stages of learning that roughly showed how problems were diagnosed, inferred about, and then solved, including the actions and interactions with the programmers and the tool sheet, programming software, and CNC machine. We reanalyzed the other 11 cases to understand what knowledge and skills the machine operators were using to diagnose problems and to solve problems independently and with the help of programmers. We rethought and

refined our three-stage model of learning based on this analysis. At this point we had a strong empirical notion of what the machine operators were learning and how they were learning.

The challenge at this stage was to both frame and theorize these findings. We read the literature on skilling and deskilling, boundary objects, epistemic objects, technology dependencies, and so on. Through iteration with the literature and engagement with the reviewers' comments, we came to see that the source of these errors that provided learning opportunities was related to the literature on representations and the problems of aligning representations with the physical phenomenon they represent. This lead us to consider the computational work involved in fundamentally realigning representations. From this theoretical perspective, and based on feedback from reviewers, we began to conceptualize both the learning process and learning outcomes we observed as related to learning computational skills and learning to be literate, at least partially, with a computational work and system.

FINDINGS

The usual and highly standardized work processes of operators were disrupted either by a machine operator's error or by factors beyond the operator's control. In the latter case, the problems were revealed when operators, having correctly followed their instructions, produced work that did not conform to information on the tool sheet and drawing. Either the pieces had extra markings, absent markings, or misplaced markings (See Appendix Table A1, examples 1, 4, 5, 6, 9, 10, and 13; Appendix Table A2, examples 5 and 10 in Additional Materials); the dimensions of the raw material did not conform to expectations (See Appendix Table A1, examples 2 and 3; Appendix Table A2, examples 8, 9, and 11 in additional Materials); or the tools did not work as expected (See Appendix Table A1, examples 7, 8, 11, and 12; Appendix Table A2, examples 1, 2, 3, 4, 6, and 7 in Additional Materials).

When problems were not the result of execution errors, programmers had to rethink the machining process and then adjust the programming script (computer code) for the mold. Operators then uploaded and ran the new program. These kinds of errors held instructive potential because they required the programmer to examine how they conceptualized the work process, identified the material variation, and reconceptualized the work given that variation. Frequently, programmers did this work with the operator sitting in their office or by going down to the shop floor. As the programmers worked, they often talked through the problem and ways of solving it.

As machine operators observed and listened to programmers attend to such errors, they developed and, over time, accumulated a set of skills that build to what we refer to as practical computational literacy. Practical computational literacy is the capacity to visualize and talk about physical objects and processes independently of them; to translate this noncomputational thinking and talking into computational symbols, syntax, structure, and assumptions; and to use this knowledge to create computational solutions that realign representations and the physical world. It is developed through the vicarious acquisition and layering of three skills. These skills are built one at a time and accumulate to a rudimentary yet practical form of computer literacy that allowed machine operators to decrease the computational opacity of representations of their work. The first skill is the capacity to *visualize* the misalignment between the representation—the natural language and assumptions of the tool sheet—and the machine and its operation. The second skill is the capacity to *translate* the features of the representation—the natural language on the tool sheet—into the symbols, syntax, and structure of computer code, and vice versa. The third skill is the capacity to *edit* the symbols and structure of computer code. We show in the next section how these skills were acquired vicariously and how, as they accumulated, operators

became increasingly capable of autonomously identifying—in noncomputational terms—problems and solutions, and of creating computational solutions. Following these, we examine the conditions under which these skills were built, and the consequences of practical computational literacy skills. Table 3 and Figure 2 summarize how operators learn to visualize the links between the representation and the physical output and how they use this skill.

Insert Table 3 and Figure 2 about here

Breaching Computational Opacity

In this section, we show how misalignments between the physical world and representations gave machine operators opportunities to observe and listen in as programmers realigned the representation. We unpack what is learned and how it is learned. We then show how it is used independently by machine operators. To facilitate understanding we include Table 4, which provides photos of the physical entity (CNC machine), the transformational technology (coding software), and the representation (tool sheet).

Insert Table 4 about here

Visualizing misalignment between the physical entity and the natural language and assumptions of the representation. Jerome (See Appendix Table A1, example 4 in Additional Materials), a machine operator, compares the finished mold with the designer drawing. The mold should have one level of indentation, but there are two. Jerome rechecks his work, going step by step through the tool sheet. Eliminating errors in execution, he takes the mold to the office. He explains the deviation between the mold and the designer drawing to David, the programmer. David opens the programming interface on his large screen. Jerome pulls up a chair and watches David click on each step of programming. David verifies that he has selected the correct tools. He explains that "everything is programmed correctly" and then plays the machining simulation, showing Jerome how two tools repeat over the same area. The role of the first tool is to change

the shape of the aluminum. The role of the second tool is to smooth and finish the shape made by the first tool. David thinks it is possible that the second tool might be shorter than expected and therefore did not reach deep enough to remove the second indentation. Moving his finger around the mold, he mimics the motion and path of the tool, showing that the shorter tool does not reach deep enough, leaving two indentations.

The machining simulation and hand gestures that mimicked the machining process allowed machine operators to fully visualize the work process that produced the machined piece. Although machine operators were able to watch the machine work through the window of the CNC machine,[3] and understand the role of each tool, the tools moved quickly along the same motion and path. Therefore, it was difficult to see the subtle moves that each tool makes to the mold and how they differed. Watching the simulation and hand gestures allowed David to see the slow motion of the work of each tool and thus the minute differences in the way each tool moved and their effect on the mold. More importantly, watching David mimic the error helped Jerome visualize the misalignment between the motions of the machine and the step-by-step instructions on his tool sheets. Visualizing the path and motion of tools and the ways in which they individually and collectively made effects allowed machine operators to assess the work process in order to identify potential problems or ways to make it more efficient.

Back on the shop floor, Jerome studies his tool sheet. After observing the machining simulation, he now understands the markings in the last column. Machine operators are instructed to use four columns of the tool sheet (Diameter, Length, Type of tool, Tool holder number) to select the correct tool and to position it. They are instructed to ignore the last column.

[3] We thank Reviewer 1 for pointing this out. In our setting, the majority of plastic machine windows were blurred because of damage from shavings, making it impossible to see the motion of the machine on the shop floor.

Through interactions such as the one presented in this vignette, machine operators learned how to interpret this column, after watching machining simulations. By comparing the machining simulations with the texts in the last column, they learned how to interpret the path and motion information in the column. Furthermore, going forward, they were able to interpret this column without consulting the machining simulation.

By observing simulations of the machining process, machine operators also learned about the constraints or assumptions embedded in their tool sheets. The representation contains assumptions, that when not reflected in the materials (exact measures) and tools (exact length), create problems. In the example above, by visualizing the tool's order, motion, and path, Jerome learned that there are some fixed assumptions inside the program (such as tool length), and he understood that any deviation from these assumptions is consequential.

Jerome measures the tools as instructed by David. The first tool is the correct length. The second tool is one millimeter shorter than expected. He comes back to programmers's office and acknowledges that David was right. David opens the CAM software. Jerome watches as David navigates the software to find where the commands related to the second tool cutter are in the programming script. Next, he copies and pastes this segment into a new folder that he saves and sends to the central computer on the shop floor. Jerome goes to the central computer on the shop floor to load this new program to his machine. He then goes to his machine, modifying the length of the tool to fit with assumptions about its length. He launches the machine. A few minutes later, the machining is finished, the problem is fixed.

In another example (See Appendix Table A1, example 2 in Additional Materials), Damien, a machine operator, struggles because the raw material is not smooth and uniform as indicated on the tool sheet. He goes to the programmers' office with the tool sheet. He explains

to Gérard, a programmer, that in some places the raw material is nearly one centimeter thicker than expected on the tool sheet. Gérard looks at the tool sheet and explains that this may cause a problem at the step on the tool sheet where the command “poche” appears in a column. He explains that the distance between the raw material and the tool holder may cause a collision which would create significant damage not only to the mold but to the machine as well. He opens the program at this specific step and launches the machining simulation. He then mimics how the machining will proceed with a raw material thicker than the one assumed in the program. With one hand representing the irregular aluminum block and the other hand representing the tool holder, he explains to Damien each of the tool’s motion and path around the aluminum. He is almost certain that there will be no collision although the distance between the raw material and the tool holder is minimal. However, to avoid a collision, he suggests decreasing the machining speed. Damien returns to the shop floor to launch the machine with a lower machining speed. In this example, Damien visualizes how the machining will proceed and learns how a specific shape is created. He also learns to visualize how a deviation from programming assumption may lead to potential collision. In Table A1 (see Appendix Additional Materials) summarizes other examples of machine operators learning to visualize their work.

Visualizing and narrating the path and motion of tools and the ways in which they individually and collectively exert effects allowed machine operators to conceptualize and assess the work process, identifying potential problems or ways to make it more efficient. For example, Jeremy reads his tool sheet slowly and carefully (Table A2, example 10). He focuses on the last column to imagine the motion and path of the tools. The designer drawing indicates that a deep hole must be machined. To make the hole, a tool that makes a small indentation should begin, followed by a tool that makes the hole deeper. However, the tool sheet indicates the reverse

order. He asks, “Why is he [the programmer] doing it like that?” He visualizes the reverse order, concluding that the second tool will vibrate and damage the mold. He concludes that there is a problem with the order of the tools and goes to talk with the programmer. The programmer runs the simulation and confirms the problem Jeremy anticipated. Once operators have learned how to read and interpret the last column of the tool sheet, they are able to reconceptualize their work. This new capacity benefitted the organization, as machine operators were able to anticipate potential problems or imagine a better work process. Table A2 summarizes other examples of machine operators using this new insight and capacity.

Learning how to visualize misalignment between each step on the tool sheet and the motion of the machine breaches computational opacity as it reveals new information implied by the tool sheet. The operators observed how these instructions—in natural language—produced an effect on the machining process. This capacity enabled them to talk through—via the instructions on their tools sheet—what the motions will be and to flag problems in the coordination between these steps to create the desired effect or outcome. They also observed that the instructions do not allow for variation in the materials or the tools. Assumptions about the dimensions and uniformity of materials and tools are embedded in the instructions, and so any deviations required consideration.

Translating the features of Representation into the symbols, syntax, and structure of the Transformational Technology. Machine operators also observed programmers working within the computer code to alter representations of machining work in relation to the presenting material conditions. Machine operators, observing programmers working within the lines of code, learned to translate the symbols, syntax, and structure of the code into the natural language

instructions on their tool sheet. These moments of translation continued to breach the opacity of the computational basis of their work.

For example, Hector, a machine operator, launches a machine but it does not respond (See Appendix Table A1, example 7 in Additional Materials). He asks Stephane, programmer, to help him on the shop floor. Stephane opens the programming script inside the machine, looks at it, and thinks aloud, “What’s wrong? Why aren’t you working?” He examines the code line by line, tracing each line with his finger as Hector looks on. Stephane points to the symbol T02M06 in the line of code: “Here the program indicates that the milling cutter will be selected.” He points to T02 to indicate the milling cutter and to M06 to indicate that it “will be selected.” Stephane then points to where T02 appears on Hector’s tool sheet. In doing so, he shows Hector the correspondence between the line of code and the line of instruction on the tool sheet. He also shows how the symbols in the programming script, indicating tools and locations, correspond to the symbols on the tool sheet.

These quick moments of connection between the unfamiliar code composed of symbols strung together and the familiar tool sheet composed of sequential concrete actions gave machine operators insights into the relationships between these seemingly disparate objects. First, they saw that the code and their tool sheet overlapped symbolically. Symbols on their tool sheet that they knew how to translate also appeared in the programming script. Because they understood these symbols on their tool sheet, they realized that they could interpret symbols in the programming script. For example, they knew that T02 referred to the milling cutter number 2. Through observing the programmer, they learned that the symbol also appeared in the programming script. Hector also learned that the symbol M06 meant “picking the tool.” Second, machine operators began to see some aspects of syntax. In this case, Hector learned that putting

two symbols together—T02 and M06—created an action, causing the machine to pick up a particular tool. Third, Hectors saw, structurally, how the lines of code mapped to lines on the tool sheet. What Hector saw specifically is that one line on the tool sheet was connected to a block of code. This was possible because Hector could—moving between his tool sheet and the code—find the tool symbol on his tool sheet and see that it identified the beginning of the block of code related to the work the tool would do. Then, the next tool on the tool sheet identified the next block of code related to the work it would do. Having knowledge of the links between their tool sheet and the code, operators began to see how the code is organized or structured in blocks. Although they did not know much (yet) about what was happening in a particular block, they understood that it pertained to the actions of a tool.

Once machine operators began to translate between tool sheet and the computer code, they discovered further connections on their own. For example, Pierre-Olivier (Table A2, example 7) compares a mold he has just finished machining with the designer drawing and sees that a hole is missing. After checking that he has made no errors, he analyzes the tool sheet. His finger traces the steps in the last column: everything is fine. He then looks up the programming script and compares it with each line on the tool sheet. He identifies that a block of code is missing. He confirms that the machine has not been instructed to make the holes. He goes to the programmer's office and tells Lionel, the programmer, "In phase 1, I should have a hole in the square pockets and there is nothing in the program." Lionel asks, "In phase 1?" while turning to his computer. He spends some time reviewing these programming choices, confirms the fix recommended by Pierre-Olivier, and then changes the program. He tells Pierre-Olivier, "It's good." Pierre-Olivier returns to the shop floor.

The skill of translating allowed operators to understand the code's symbols, syntax, and block structure. They read through the code in a very rough, practical manner, looking for symbols they understood, understanding simple syntax, and navigating the structure. They identified important touch points between their work instructions and the code itself, opening the code and making it accessible to them. This, as the example of Pierre-Olivier shows, created the capacity to detect and anticipate problems but also to suggest solutions. Furthermore, using their tool sheet, machine operators toggled between the motions of the machine (visualizing) and the code instructing the machine (translating). While these two skills did not allow operators to change the computer code, they allowed them to identify errors in the computational language (omission or wrong symbols, syntax, or block of codes) and provide advice to programmers about the segments of code needing revision.

Editing the symbols and structure within the transformational technology to create new physical entities. Machine operators, observing programmers working within the lines of code as they solved problems, also learned to edit the code. Continuing example 7 (See Appendix Table A1 in Additional Material) from above, Stephane continues to study the screen with Hector looking on. It is not clear why the machine will not pick up the milling cutter (T02) to execute the first cut on the aluminum. Stephane decides to choose an identical milling cutter located in a different position in the machine. He returns to the line of code, deletes the 2 in T02 and enters 5, which indicates an identical cutter (T05) in another tool holder. He saves the program and starts the machine. It works.

Watching Stephane, Hector had the opportunity to learn how to edit the programming script on his machine. He did not learn how to program, but he learned how to delete a symbol and enter another one. This was a small but significant change that Hector learned how to

perform. The programming script was not only increasingly open to Hector but also increasing pliable. Although the programming script was deeply embedded in the CNC machine, once operators learned how to translate and edit the programming script, they could search for this script in order to manipulate it independently.

This editing capacity was used creatively by machine operators. It was used, as demonstrated above, to delete and insert new tool codes. It was used to copy sections of code and to save them as a separate script in cases where machine operators wanted to repeat some actions. For example, Alexandre needs to detach the outer edges of the mold (Table A2, example 1). The tool length deviates by one-tenth of a millimeter, so it does not cut deeply enough to detach the outer edge. To solve this problem, the machine needs to recut the outer edge only. He opens the programming script on the shop-floor computer and begins to read the lines of code onscreen, comparing them with the lines on the tool sheet. In so doing, he figures out which block of code refers to the trimming of the edge. He copies and pastes this block, saves it under a new job number, and uploads this subprogram to his machine. Once he has positioned the tool slightly lower, he reruns this subprogram to detach the mold. Because the machine operators understood the code's structure, they could cut and paste to edit the parts of the programming script. Note that they did not edit syntax. They cobbled together their insights into the programming script to make small changes to the script, to copy and move it, and to combine pieces of script. Other examples of machine operators editing the programming script or asking programmers to edit the programming scripts are summarized in Table A2.

Machine operators who acquired this editing capability provided important benefits to the organization. First, as in the example with Alexandre, they edited programming scripts independently to fix problems on the shop floor. Second, as in the example with Hector, fixes to

programming errors were saved, creating a new programming script related to a job. Because programmers reused and repurposed programming scripts, this change improved the quality of the code for future jobs. This skill had important short- and long-term benefits for the organization.

The Vicarious Learning Process

Machine operators developed practical computational literacy through observing programmers fix problems related to programming errors or material deviations. The examples above and in Table A1 (see Appendix Additional Materials) show that as programmers went about fixing problems, they narrated their work process. Sometimes machine operators were addressed (See Appendix Table A1, examples 1–6, 9–10, and 13 in Additional Materials). Other times they were ignored (See Appendix Table A1, examples 7, 8, and 11 in Additional Materials). In either case, machine operators watched as programmers played and replayed machining simulations and mimicked the machine movements to find errors. They watched and listened as programmers moved between the natural language on their tool sheet and the programming script. They observed how changes to the script were made. Here the programmers were not "modeling" their full skill set or role. They were unintentionally modeling how to fix problems by reconceptualizing and reprogramming, often in minor ways, the work of machine operators. The machine operators watched, listened, and learned. They collected and stored bits of insight about the assumptions and information embedded in their tool sheets, how the tool sheets could be translated into code, and how the code could be edited.

The Conditions for Accumulating Practical Computational Literacy Skills

These moments exposed machine operators to the abstract thinking and computational symbols, syntax, and structure that inform their work and were facilitated by several factors. First, they were triggered by problems requiring machine operators to seek the help of

programmers. These non-execution problems – misalignments between the representation and the physical world – were occasions for machine operators to learn because they required computational fixes. Thus, they were opportunities to learn about how their work was conceptualized and programmed by programmers. Second, machine operators required opportunities to observe programmers fix these problems "aloud", meaning not only did programmers work on the problems they did by this talking, gesturing, and otherwise revealing their diagnosis and solution process. This talk of the computational work occurred as the programmer thought through the problem at hand.

Third, relatedly, opportunities to develop practical computational literacy skills existed because programmers did not perceive their positions or authority as challenged or threatened by their interactions with machine operators around these problems. The programmers were not observed to be protective of their knowledge. This is in part because the divisions between the groups were respected and enforced through job roles, employment contracts, and local norms strongly entrenched within the French regulatory context. Fourth, machine operators were able to learn and use practical computational literacy skills because they could access the programming script related to their work. Learning how to translate and edit required that machine operators be able to open, examine, and make changes to this script. We discuss the generalizability of these conditions in the discussion section.

Not all machine operators developed practical computational literacy. Of the 17 machine operators observed, 6 did not demonstrate practical computational literacy. Of the remaining 11, 7 operators, with tenure between 1 and 35 years, demonstrated all three levels of practical computational literacy. The remaining 4 demonstrated two levels. This achievement of practical

computational literacy indicates that it was not a straightforward skill to achieve and that individual differences in capacity and interests mattered to some extent.

The Consequences of Practical Computational Literacy

Consequences of practical computational literacy for machine operators. Practical computational literacy allowed machine operators to work autonomously as they reconceptualized their work on the shop floor, identified problems, and posed computational solutions. It allowed them to transcend their routine work temporarily and to exercise discretion periodically within the production process. When using this skill, operators were able, momentarily to think and work like programmers. They conceptualized and reconceptualized their work, and they solved problems stemming from variations in materials or from errors in the conceptual work of programmers autonomously. As the opacity of the computational processes informing their work decreased, operators also brought changes and suggestions to the programmers. In example 1 (Table A2), Alexandre visualizes and translates. He identifies where in the physical process the problem comes from and locates the block of code causing the problem. He has never edited code on his own; however, given his capacity to visualize and narrate his work and translate this to and from computational symbols, syntax, and structure, he feels comfortable trying. He copies and pastes one block of code. This experimentation allows him to acquire the third level of practical computational literacy: editing. As machine operators experimented with these kinds of small changes, they expanded their abstract thinking, their knowledge of the symbols, the syntax and structure of the code, and their capacity to diagnose and solve problem.

In Table A2 (examples 1–6), we summarize examples in which operators use practical computational literacy to work autonomously and make decisions about how to reconceptualize their work. In some cases, machine operators went beyond solving problems themselves. They

used their practical computational literacy to explain to programmers how they should revise the representation to solve the problem (Table A2, examples 7–11). For example, Laurent is about to launch a program to machine a very large mold (Table A2, example 9). He seems skeptical about how this will be done. He reads the last column on the tool sheet slowly. He goes row by row, talking through the instructions while touching the mold. He analyzes how the various tools will move around the mold and where the tools will cut. He realizes that it is possible to do the machining by turning the mold 90 degrees. This will make it easier to do the job, but a new representation of the work will need to be created. He goes to the programmer to discuss and request the changes. The programmer agrees to make the changes. This example illustrates how the use of practical computational literacy allowed machine operators to think like those who created the representation. In this way, there was a momentary shift from the operator's role as executing the manual part of the process to conceptualizing it. They could, at least temporarily, instruct programmers. They also gained some autonomy by choosing how to improve conceptualizations of their work.

The machine operators who were practically literate in computational terms were identified by supervisors and programmers as higher-skilled. Although the particular knowledge and capacities that constitute practical computational literacy were not formally recognized or rewarded by the organization through an increase in salary or job level, supervisors informally recognized those who acquired this skill and considered them to be very good machine operators, assigning them the most complicated molds to machine. For example, some tool sheets had more than forty lines of instructions, and a single conceptual error might damage or even destroy the mold. To avoid potential problems, managers ensured that such molds were machined by machine operators that we observed to have practical computation literacy. Machining these

molds provided machine operators the opportunity to cope with their routinized work; as explained by Laurent, when working on a complex mold, “I love doing stuff like that, it’s so interesting.”

Among machine operators, those who were computationally literate were described differently. They were known as workers who could solve problems and teach others how to solve problems. For example, François, encountering a deviation between the instructions and the physical material, says, “Let’s talk to Laurent: he will know how to fix it, he knows everything.” Roland describes his colleague: “Fortunately there is Jean. He is so smart. He explains everything to me.” These machine operators were consistently referred to as “smart,” “very good,” “good at the job,” or “know[ing] what they are doing.”

Consequences of practical computational literacy for the organization. Operators who used their computational literacy on the shop floor generated benefits for the organization, as they facilitated the smooth functioning of the production process by fixing problems (Table A2, examples 1–7), avoiding problems (Table A2, examples 10 and 11), or improving programs (Table A2, examples 2 and 4–11). For example, Jean (Table A2, example 5) is able to fix problems on a mold that was damaged during a previous machining. He opens the programming script inside the CNC machine, looks at it, and begins to edit it. At that moment, Gérard, a supervisor, arrives and sees Jean making changes to the programming script. He pretends not to look at him and says to the first author, “He is really good. That’s why I can let him do that,” and he returns to his office. Practical computational literacy benefits the organization, as machine operators can fix problems without the help of the programmers.

Computational literacy also helps operators avoid problems**.** For example, before installing the material in the machine, Jacques moves his hands in waves around the tool sheet.

He explains what he is doing: “I always imagine the tools’ motion and path before starting the machining.” In this imagining process, Jacques anticipates and avoids problems before starting the machine. Using practical computational literacy before starting a machining allows François (Table A2, example 10) to anticipate potential damage to the mold. By reading the tool sheet, he is able to visualize how the order of tools will create vibrations that may damage the mold.

Operators also used practical computational literacy to improve programming scripts. In examples 2 and 4 through 11 (Table A2), machine operators suggested or made modifications to the code to fix errors made by programmers. These improved programs were then saved. Later, programmers reused or repurposed these programming scripts when working on similar jobs. The changes suggested or made by operators had longer-term benefits for the organization, as they corrected errors in the code and increased the quality of the programming scripts.

DISCUSSION

Representations are increasingly important to instructing, mediating, or replacing work with physical objects and processes. They inform the work of physicians (Lebovitz, Lifshitz-Assaf, & Levina, 2022), investment bankers (Beunza & Stark, 2012), pharmacist assistants (Barrett et al., 2012), and shop-floor workers (Bechky, 2003a, 2003b). However, the computational work that generates these representations is a black box for most workers. The computational opacity of the representations is consequential when misalignments between these depictions of the physical world and the physical world itself occur. When misalignments occur, we depend on programmers to reconceptualize and reprogram representations. However, such moments may also be transformed into opportunities to learn—vicariously—from programmers about the computational work required to transform the physical into a representation and the coding work involved in these transformations.

In our setting, workers took advantage of instances of misalignment to develop practical computational literacy skills. We use the term literacy to refer to a capacity to comprehend—read, make inferences, and reason about—a system of symbols organized around particular logics and rules (Farrar, 1986). We use the term practical to refer to the limits of the literacy observed and the way in which the machine operators' literacy skills emerged and developed. Practical or functional refers to a degree of literacy focused on what is useful for and effective in problem-solving work. At the same time, these skills have a general quality, beyond the problem at hand, as they created a budding comprehension of a system of symbols, and the logics and rules through which they were organized. We observed that machine operators continued to build and develop this comprehension with respect to specific problems and to machining work in general. Thus, it is possible that the practical quality of the skills might evolve toward more sophisticated work within this system of symbols.

The computational component of the skills refers to the machine operators' ability to see their work through the eyes of programmers. The operators' understanding of the machining processes became more abstract, moving from natural language (first level) to computational language (second level). They advanced from talking and thinking about problems and solutions in natural language to talking and thinking about problems and solutions in computational language. They also learned to move fluently between these languages. Finally, they learned how to implement their solutions by editing the code, cutting, copying, and pasting the symbols and blocks of code. Observing how their work was conceptualized, and converted into code, operators could reflect on and rethink it when problems occurred. Practical computational literacy is evidenced by the ability of machine operators to transform the given representations of their work.

Practical computational literacy is learned vicariously as members of one occupational community observed the work of another occupational community. The programmers were not teaching or attempting to transfer their experience and skills to machine operators. Machine operators simply observed and listened to programmers work. As the programmers worked, they shared knowledge and skills that were mundane to them but new and interesting to the machine operators. The machine operators collected these tidbits of information. As they observed and interacted with programmers they came to understand the computational work embedded in their representations. Such learning is different from situated learning through legitimate peripheral participation, as operators did not learn programmers' job, how to use programmers' tools, or how to generate representations. However, this vicarious learning gave operators the opportunity to go beyond their job and fix problems computationally.

Practical computational literacy is generative, in that it creates the capacity to teach and build this skill in peers. Workers who have developed this capacity can continue to expand their role by passing their knowledge on to other workers. Beyond the challenge of learning, the machine operators experienced momentary autonomy and discretion, and the possibility of teaching the skill to peers. In our setting, these skills brought a strong positive reputation to machine operators on the shop floor but did not lead to broader or more formal rewards and recognition. In other settings that might not be the case. These skills bring considerable benefits to organizations, even though managers may be unaware of them, as misalignment problems are avoided and fixed, and work processes are improved.

CONTRIBUTIONS

Understanding Representations as Computationally Opaque

Our study foregrounds and examines an important form of opacity that is likely to become more prevalent and consequential within organizations. Computational opacity refers to

the black boxed computational work that generates the myriad representations that are increasingly central to work of all kinds. Representations are most often studied as objects containing and revealing information that makes organizational processes, actors, and opportunities visible and knowable (e.g., Carlile, 2002; Leonardi & Bailey, 2017; Leonardi & Treem, 2020). Thus, representations serve as boundary objects (Star, 1989), epistemic objects (Knorr-Cetina, 1999), and knowledge objects that contain and have the potential to impart and generate knowledge (Alaimo & Kallinikos, 2022). However, representations also conceal the computational work used to transform data about the physical world into abstract depictions. Representations are computational opaque for most other workers because the skills to read and write code are the domain of specialists such as programmers, software engineers, and developers. They are both closed – workers cannot access the coding logic and coding script generating representation – and unpliable – the representations cannot be modified or updated. Computational opacity is of consequence when representations need to be altered – reconceptualized and reprogrammed – to more accurately reflect the phenomenon they depict. For example, when art historians encounter algorithmic recommendations that misclassify art works, they rely on programmers to "repair" the algorithm so that recommendations are more accurately aligned with the appearance of the art (Sachs, 2020). When pharmacist assistants notice a misalignment between the computer inventory and the physical inventory, they depend on a programmer to fix the problem (Barrett et al., 2012). Technical illiteracy – the inability to read and write code – creates a dependence on those with computational skills—programmers, developers, computer scientists (Burrell, 2016). Because these skills are in demand, such a dependence creates bottlenecks and inefficiencies when representations need to be computationally altered.

Breaching Computational Opacity

Beyond, conceptualizing and problematizing this form of opacity, we identify an alternative means of breaching it. Typically, when workers encounter a misalignment between a representation and the physical world, they workaround the computational opacity of the representation. The literature documents two types of workarounds. Workers may alter their interpretation of the representation without changing the programmer's conceptualization or coding that continues to produce representations (e.g., Anthony, 2021). Alternatively, workers may alter the physical world to align with a representation based on the knowledge and skills of a community of practice (e.g., Juravich, 2017; Merschbrook & Figeres-Munoz, 2015). In either case, the computational thinking and coding responsible for the misalignment is not altered. These workarounds temporarily solve the misalignment problem without solving the fundamental computational problems. Further, they do not provide workers any insight into the computational work and the computational symbols, syntax, and structure that create the representations. This suggests that rather than improving representations, organizations may work based on one-off patches, changes, and fixes, which raises issues of accuracy, efficiency, and quality of organizational goods and services.

Practical computational literacy, the capacity to read and write code in a limited yet functional way, is an alternative means of managing misalignment by addressing the computational source of the misalignment. Because these skills reduce the computational opacity of representations, attainment of these skills allows workers to fix—in fundamental and enduring ways—the problems they encounter in their work related to the computational thinking and processes that inform that work. Practical computational literacy allows worker to fix misalignment problems at the point of production, thus workers experience momentary autonomy and discretion in both conceptualizing and programming aspects of their work. They

also develop a new capacity that they can continue to build and expand to anticipate and avoid problems and improve work processes. Practical computational literacy is a versatile capacity that can be employed to improve work processes more generally and to explore other applications of these skills in the workplace. Workers can proactively use these skills to understand other computational inputs into their work and how they can be edited.

Learning Practical Computational Literacy

Most contemporary understandings of learning in organizations draw on the concept of situated learning (Lave & Wenger, 1991). Situated learning happens most often through legitimate peripheral participation, meaning "the process by which newcomers become part of a community of practice" (Lave & Wenger, 1991: 29). For example, apprentices in a glass-blowing company learn their work with senior craftsmen who show and teach them how to execute the work (O'Connor, 2007). That is, entering or novice members of the community are given the opportunity to observe the work of seniors and to participate peripherally in the work as they learn the required knowledge and skills. Even within these communities, novice members may be denied opportunities to learn (Beane, 2019; Bharatan, Swan, & Oborn, 2022) and must overcome temporal (O'Mahony & Bechky, 2006), spatial (Beane, 2019), and hierarchical (Kellogg, Meyer et al., 2021) boundaries.

Thus, the barriers that prevent outsiders from learning the work of an occupational community seem insurmountable. Our paper contributes to an emerging stream of literature that examines how outsiders to a community learn the skills of that community. Kaynak (2023) shows how non-computer-science graduates who aspire to coding jobs learn through peer and near peer project work at boot camps. We show how practical, useful aspects of one occupational community's knowledge and skills can be developed vicariously by members of another occupation. In doing so, we suggest a learning pathway, although partial, that may be more

available in organizations but that has not received attention. As members of one community of practice work with members of another community of practice to work on hands-on projects or problems, there are opportunities to observe and listen to how work is done. The ways of thinking, the language, and the techniques of other communities of practices can be collected and used. This may happen relatively frequently; however, when it happens in the context of specialized technical skills, the vicarious learning becomes more apparent. Our study suggests that vicarious coding is a learning pathway in organizations that requires further attention.

Our paper also offers contributions to the literature on vicarious learning. Vicarious learning is commonly examined as the process through which members of an occupational community learn from their peers by observing and imitating them, "with the learner motivating his or her own behavior toward alignment with that of the model" (Myers, 2018 p.610, citing Bandura, 1977a). For instance, nurses successfully navigate challenging situations by emulating the behaviors of other nurses who have prior experience with such situations (Roberts, 2010; Myers, 2022). Similarly, during the COVID-19 pandemic, doctors observed and imitated strategies employed by doctors in countries and hospitals that had more experience treating the virus to effectively address this critical and unexpected crisis (Myers, 2020). Negotiators improve their techniques by recognizing and emulating effective negotiators (Nadler, Thompson and Van Boven, 2003). However, we show vicarious learning occurring across occupational groups, thereby taking up Myers' (2022) suggestion to examine the potential extension of vicarious learning beyond within occupational exchanges.

In documenting vicarious learning across two occupational groups, we also note the importance of translation processes in such contexts. Knowledge in one occupation is often not readily applicable for someone else's learning in a different context (Bailey, Leonardi, and

Barley, 2012). Thus, similar to Bresman (2013) we observe how vicarious learning goes beyond a "find and copy" process to require the translation of practices from one occupational community to another. This involves moving from observational learning to experimental learning (Bingham & Davis, 2012). Through this process workers “make meaning from another’s experience” (Myers, 2018: 610), going beyond imitation. In our study, machine operators work out which the bits of programming knowledge and technique are relevant to the problem at hand and how to apply them, translating observations of the practices of other occupations into their own occupational practices.

Boundary Conditions

Both the skill learned and the learning process suggest that particular conditions—misalignment problems that require a computational fix, observation of the work of programmers, and access to the programming scripts (computer code) that generates the representation—are needed. However, these conditions are increasingly possible. As the literature and our data show, misalignment problems are encountered regularly. Furthermore, while direct observation of the work of programmers might not be physically or temporally possible as in our setting, resources that facilitate learning about this work is increasingly available online. Those without formal training who are trying to break into coding work can learn and develop their skills by consulting StackOverflow (question and answer forum), blog posts, video tutorials, and GitHub (public code repository) (Kaynak, 2023). Together, these publicly available resources represent a crowdsourced body of programming knowledge. They are a virtual way of listening to programmers talk and observing them discuss and fix problems. For example, as novices encounter problems when attempting to build their own websites, they consult blogs to figure out how to solve problems (Pillai, Shinohara, & Tigwell, 2022). Finally, access to the programming scripts that generate the representations we work with are also

increasingly available or findable using the online resources mentioned above. For example, if one wants to understand how to access the programming script that informs a 3D printer or home assistant applications, an internet search provides several guides to accessing the code. Likewise, problems with email may inspire a web search that shows a non-programmer how to find and adjust the relevant code (Michail, 2002).

Our findings suggest that the capacity to develop practical computational literacy is not constrained by the domain or type of work. We expect that if the conditions articulated above are met, lawyers, radiologists, pharmacy assistants, and bankers could develop the skills to read and edit the code that generates the representations from which they work. The main constraints are vicarious access to computational logics and methods, and the transformative technologies. However, where technologies are less scrutable or explainable, such as learning algorithms, building practical computational literacy will be limited. Programmers may have the capacity to modify the algorithm; however, they may not understand the logic and inner workings of the computational engine. Thus, they may alter the representation but cannot predict whether it will create better alignment with the physical world represented. That is, the representation is computational pliable but closed. For these reasons, vicarious coding is possible but limited in the case of learning algorithms because the computational logic cannot be observed.

Future Research

Computational opacity has been taken-for-granted as part and parcel of the vertical division of labor. Those who design and create representations work upstream from those who, with narrowly defined jobs and skill requirements, work from representations (Zetka, 1992). Representations are meant to provide instructions and specifications that guide and constrain the work of those who do routine call-center work, production work, clerical work, or kitchen work. Representations are understood to create codified work, scripted responses, mindless jobs, and

controlled workers (Braverman, 1974). Thus, the computational opacity of representations is intended to maintain control over the workers and work done from representations. In those cases, the fact that workers lack computationally literacy is not necessarily a problem and can even be perceived by manager as an opportunity to maintain control over them and the labor process.

However, as the division of labor is increasingly horizontal, workers may have both motive and opportunity to breach computational opacity. Computational opacity is increasingly an issue for lawyers (Kluttz & Mulligan, 2019), bankers (Anthony, 2021), traders (Beunza & Stark, 2012), and human resource managers (van den Broek et al., 2021). As new analytical technologies create predictions and recommendations, drawing on some combination of the profession's conceptual knowledge and data of the physical realm in which the profession works, these representations of legal discovery summaries, financial calculations, recommended buys/sells, and recommended hires are opaque and inaccessible to the profession that works from them. Understanding and controlling the computational bases of such work, may become increasingly important for professional and occupational groups as their work becomes more reliant on computational work and as computational workers remain in short supply.

This suggests the need to study how occupational communities and professions are adjusting to the ongoing digitalization of almost all aspects of work. Beyond workarounds (Lebovitz et al., 2022; Anthony, 2021), is the demand for computational skills increasing? How is this reshaping the core work and knowledge of the occupation or profession? Is it possible that such skills will be increasingly required as a complement to substantive domain skills and knowledge? Is there variation in how this demand is being addressed? Are new hybrid roles emerging within occupations and professions that combine knowledge of the substantive and

computation work? Do such roles emerge through local, informal dynamics or do they rely on institutional changes to educational and training programs? Or, are new forms of collaborations emerging with programmers, developers, and computer scientists? Future research should investigate how the increasing relevance of computational opacity is changing skills and roles within occupations and professions.

The development of practical computational literacy may create negative consequences for organizations and control challenges for managers. Vicarious coding suggests that there may be more workers who can code than the designated programmers and developers. In some settings, vicarious coding may intentionally or unintentionally lead to unproductive or damaging outcomes. For example, editing the code in any tightly coupled system is likely to lead to a range of negative disruptions or even disaster. As result, managers need to be aware of the potential for workers to develop the capacity to code vicariously and consider the consequences of distributed, local changes to code. Paying attention to this skill development, will allow managers to leverage this skill to avoid and solve problems. Future research may examine the degree and distribution of coding competencies in organizations and their consequences for organizational performance. These coding competencies are likely to be related to the degree of openness and pliability of the representation. For this reason, future studies could examine how this variation shapes the potential for learning computational skill.

REFERENCES

Abbott, A. (1988). ***The system of professions: An essay on the division of expert labor***. University of Chicago press.

Alaimo, C., & Kallinikos, J. (2022). Organizations decentered: data objects, technology and knowledge. ***Organization Science*****,** *33*(1), 19-37.

Anthony, C. (2018). To Question or Accept? How Status Differences Influence Responses to New Epistemic Technologies in Knowledge Work. ***Academy of Management Review*****,** *43*(4), 661-679. https://doi.org/10.5465/amr.2016.0334

Anthony, C. (2021). When knowledge work and analytical technologies collide: The practices and consequences of black boxing algorithmic technologies. ***Administrative Science Quarterly*****,** *66*(4), 1173-1212.

Bailey, D. E., & Barley, S. R. (2011). Teaching-learning ecologies: Mapping the environment to structure through action. ***Organization Science*****,** *22*(1), 262-285.

Bailey, D. E., Leonardi, P. M., & Barley, S. R. (2012). The Lure of the Virtual. ***Organization Science*****,** *23*(5), 1485-1504. https://doi.org/10.1287/orsc.1110.0703

Bailey, D. E., Leonardi, P. M., & Chong, J. (2010). Minding the Gaps : Understanding Technology Interdependence and Coordination in Knowledge Work. Organization Science, 21(3), 713-730. https://doi.org/10.1287/orsc.1090.0473

Bandura, A. (1977). ***Social learning theory***. Englewood Cliffs, NJ: Prentice Hall

Barley, S. R. (1986). Technology as an occasion for structuring: Evidence from observations of CT scanners and the social order of radiology departments. ***Administrative Science Quarterly***, 78-108.

Barley, S. (1996). Technicians in the workplace : Ethnographic evidence for bringing work into organization studies. ***Administrative Science Quarterly***, *41*(3), 404-441. https://doi.org/10.2307/2393937

Barley, S. R., & Bechky, B. A. (1994). In the Backrooms of Science : The Work of Technicians in Science Labs. ***Work and Occupations***, *21*(1), 85-126. https://doi.org/10.1177/0730888494021001004

Barley, W. C. (2015). Anticipatory Work : How the Need to Represent Knowledge Across Boundaries Shapes Work Practices Within Them. ***Organization Science***, *26*(6), 1612-1628. https://doi.org/10.1287/orsc.2015.1012

Barrett, M., Oborn, E., Orlikowski, W. J., & Yates, J. (2012). Reconfiguring Boundary Relations : Robotic Innovations in Pharmacy Work. ***Organization Science***, *23*(5), 1448-1466. https://doi.org/10.1287/orsc.1100.0639

Beane, M. (2019). Shadow learning: Building robotic surgical skill when approved means fail. ***Administrative Science Quarterly*****,** *64*(1), 87-123.

Bechky, B. A. (2003a). Sharing Meaning Across Occupational Communities : The Transformation of Understanding on a Production Floor. ***Organization Science***, *14*(3), 312-330. https://doi.org/10.1287/orsc.14.3.312.15162

Bechky, B. A. (2003b). Object Lessons : Workplace Artifacts as Representations of Occupational Jurisdiction. ***American Journal of Sociology*****,** *109*(3), 720-752. https://doi.org/10.1086/379527

Beunza, D., & Stark, D. (2012). From dissonance to resonance : Cognitive interdependence in quantitative finance. ***Economy and Society***, *41*(3), 383-417. https://doi.org/10.1080/03085147.2011.638155

Bharatan, I., Swan, J., & Oborn, E. (2022). Navigating turbulent waters: Crafting learning trajectories in a changing work context. ***Human Relations*****,** *75*(6), 1084-1112.

Bingham, C. B., & Davis, J. P. (2012). Learning sequences: Their existence, effect, and evolution. ***Academy of Management Journal***, *55*(3), 611-641.

Black, P., Harrison, C., Lee, C., Marshall, B., & Wiliam, D. (2004). Working inside the black box: Assessment for learning in the classroom. ***Phi delta kappan***, *86*(1), 8-21.

Boland Jr, R. J., Lyytinen, K., & Yoo, Y. (2007). Wakes of innovation in project networks: The case of digital 3-D representations in architecture, engineering, and construction. ***Organization Science*****,** *18*(4), 631-647.

Boudreau, M.-C., & Robey, D. (2005). Enacting Integrated Information Technology : A Human Agency Perspective. ***Organization Science*****,** *16*(1), 3-18. https://doi.org/10.1287/orsc.1040.0103

Braverman, H. (1974). ***Labor and monopoly capital.*** New York: Monthly Review.

Burrell, J. (2016). How the machine 'thinks': Understanding opacity in machine learning algorithms. *Big data & society*, *3*(1), 2053951715622512.

van den Broek, E., Sergeeva, A., & Huysman, M. (2021). When the Machine Meets the Expert: An Ethnography of Developing AI for Hiring. ***MIS Quarterly*****,** *45*(3).

Brown, J. S., & Duguid, P. (1991). Organizational learning and communities-of-practice: Toward a unified view of working, learning, and innovation. ***Organization Science*****,** *2*(1), 40-57.

Carlile, P. R. (2002). A Pragmatic View of Knowledge and Boundaries : Boundary Objects in New Product Development. ***Organization Science*****,** *13*(4), 442-455. https://doi.org/10.1287/orsc.13.4.442.2953

Dean, J. W., Yoon, S. J., & Susman, G. I. (1992). Advanced Manufacturing Technology and Organization Structure : Empowerment or Subordination? ***Organization Science***, *3*(2), 203-229. https://doi.org/10.1287/orsc.3.2.203

Dodgson, M., Gann, D. M., & Salter, A. (2007). "In Case of Fire, Please Use the Elevator" : Simulation Technology and Organization in Fire Engineering. ***Organization Science*****,** *18*(5), 849-864. https://doi.org/10.1287/orsc.1070.0287

Ewenstein, B., & Whyte, J. (2009). Knowledge Practices in Design : The Role of Visual Representations as `Epistemic Objects'. ***Organization Studies*****,** *30*(1), 07-30. https://doi.org/10.1177/0170840608083014

Farrar, M. T. (1986). Four conceptions of literacy. *Reading Psychology: An International Quarterly*, *7*(1), 43-55.

Fujimura, J. H. (1992). Crafting science: Standardized packages, boundary objects, and "translation.". ***Science as Practice and Culture*****,** *168*(1992), 168-69.

Gioia, D. A., & Manz, C. C. (1985). Linking cognition and behavior: A script processing interpretation of vicarious learning. **Academy of management Review**, 10(3), 527-539.

Glaser, B. G., & Strauss, A.L. (1967). ***The Discovery of Grounded Theory: Strategies for Qualitative Research*****,** Chicago, Aldine Publishing Company

Hazlehurst, R. J., Bradbury, R. J., & Corlett, E. N. (1969). Comparison of Skills of Machinists on Numerically-Controlled and Conventional Machines.***Occupational Psychology*****,** *43*(3-4), 169-182.

Houston, L., Gabrys, J., & Pritchard, H. (2019). Breakdown in the Smart City : Exploring Workarounds with Urban-sensing Practices and Technologies. ***Science, Technology, & Human Values*****,** *44*(5), 843-870. https://doi.org/10.1177/0162243919852677

Huising, R. (2019). Moving off the Map : How Knowledge of Organizational Operations Empowers and Alienates. ***Organization Science***, *30*(5), 1054-1075. https://doi.org/10.1287/orsc.2018.1277

Jones, B. (1982). ***Destruction or redistribution of engineering skills: the case of numerical control***[in:] Wood S.(ed.): The Degradation of Work. Hutchinson, London.

Jordan, T. (2009). Hacking and power: Social and technological determinism in the digital age. ***First Monday***.

Juravich, T. (2017). Artifacts of workers' knowledge : Finding worker skill in the closing and restructuring of a furniture manufacturer. ***Ethnography***, *18*(4), 493-514. https://doi.org/10.1177/1466138116687590

Kallinikos, J. (2009). On the Computational Rendition of Reality : Artefacts and Human Agency. ***Organization***, *16*(2), 183-202. https://doi.org/10.1177/1350508408100474

Kallinikos, J., Aaltonen, A., & Marton, A. (2010). A theory of digital objects. ***First monday*****,** *15*(6), 1-22.

Kallinikos, J., Aaltonen, A., & Marton, A. (2013). The ambivalent ontology of digital artifacts. ***MIS Quarterly***, 357-370.

Kaynak, E. (2023). Breaking into Expert Practice: Occupational Learning at Coding Bootcamps. ***Organization Science,*** *Forthcoming.*

Keefe, J. H. (1991). Numerically Controlled Machine Tools and Worker Skills. ***Industrial and Labor Relations Review*****,** *44*(3), 503. https://doi.org/10.2307/2524158

Kellogg, Katherine C., Jenna E. Myers, Lindsay Gainer, and Sara J. Singer. "Moving violations: Pairing an illegitimate learning hierarchy with trainee status mobility for acquiring new skills when traditional expertise erodes." ***Organization Science*** 32, no. 1 (2021): 181-209.

Kellogg, K. C., Orlikowski, W. J., & Yates, J. (2006). Life in the Trading Zone : Structuring Coordination Across Boundaries in Postbureaucratic Organizations. ***Organization Science*****,** *17*(1), 22-44. https://doi.org/10.1287/orsc.1050.0157

Kluttz, D., & Mulligan, D. K. (2019). Automated decision support technologies and the legal profession. ***SSRN Electronic Journal.*** https://doi.org/10.2139/ssrn.3443063

Lave, J., & Wenger, E. (1991). **Situated learning: Legitimate peripheral participation**. *Cambridge university press.*

Lebovitz, S., Lifshitz-Assaf, H., & Levina, N. (2022). To Engage or Not to Engage with AI for Critical Judgments : How Professionals Deal with Opacity When Using AI for Medical Diagnosis. ***Organization Science*****,** *33*(1), 126-148. https://doi.org/10.1287/orsc.2021.1549

Leonardi, P. M., & Bailey, D. E. (2008). Transformational Technologies and the Creation of New Work Practices : Making Implicit Knowledge Explicit in Task-Based Offshoring. ***MIS Quarterly***, *32*(2), 411-436. https://doi.org/10.2307/25148846

Leonardi, P. M., & Bailey, D. E. (2017). Recognizing and selling good ideas: Network articulation and the making of an offshore innovation hub. ***Academy of Management Discoveries*****,** *3*(2), 116-144.

Leonardi, P. M., & Treem, J. W. (2020). Behavioral visibility: A new paradigm for organization studies in the age of digitization, digitalization, and datafication. ***Organization Studies*****,** *41*(12), 1601-1625.

Van Maanen, J., & Barley, S. R. (1984). Occupational communities: Culture and control in organizations. ***Research in Organizational Behavior.***

Manz, C. C., & Sims Jr, H. P. (1981). Vicarious learning: The influence of modeling on organizational behavior. ***Academy of Management Review*****,** *6*(1), 105-113.

Mazmanian, M., Cohn, M., & Dourish, P. (2014). Dynamic Reconfiguration in Planetary Exploration : A Sociomaterial Ethnography. ***MIS Quarterly*****,** *38*(3), 831-848.

Merschbrock, C., & Figueres-Munoz, A. (2015). Circumventing Obstacles in Digital Construction Design – A Workaround Theory Perspective. ***Procedia Economics and Finance*****,** *21*, 247-255. https://doi.org/10.1016/S2212-5671(15)00174-4

Michail, A. (2002, May). Browsing and searching source code of applications written using a GUI framework. In ***Proceedings of the 24th International Conference on Software Engineering*** (pp. 327-337).

Myers, C. G. (2018). Coactive vicarious learning: Toward a relational theory of vicarious learning in organizations. ***Academy of Management Review***, *43*(4), 610-634.

Myers, C. G. (2022). Storytelling as a tool for vicarious learning among air medical transport crews. ***Administrative Science Quarterly***, 67(2), 378-422.

Nadler, J., Thompson, L., & Boven, L. V. (2003). Learning negotiation skills: Four models of knowledge creation and transfer. ***Management Science***, 49(4), 529-540.

Noble, D. F. (1978). Social Choice in Machine Design : The Case of Automatically Controlled Machine Tools, and a Challenge for Labor. ***Politics & Society***, 8(3-4), 313-347. https://doi.org/10.1177/003232927800800302

O'connor, E. (2007). Embodied knowledge in glassblowing: the experience of meaning and the struggle towards proficiency. ***The Sociological Review***, *55*, 126-141.

O'Mahony, S., & Bechky, B. A. (2006). Stretchwork: Managing the career progression paradox in external labor markets. *Academy of Management Journal*, *49*(5), 918-941.

Orr, J. E. (1996). **Talking about Machines : An Ethnography of a Modern Job**. Cornell University Press.

Orr, J. E. (1998). Images of Work. ***Science, Technology, & Human Values*****,** *23*(4), 439-455. https://doi.org/10.1177/016224399802300405

Pillai, A., Shinohara, K., & Tigwell, G. W. (2022, October). Website Builders Still Contribute to Inaccessible Web Design. In ***Proceedings of the 24th International ACM SIG ACCESS Conference on Computers and Accessibility*** (pp. 1-4).

Pollock, N. (2005). When is a work-around? Con ict and negotiation in computer systems development. ***Science, Technology & Human Values***, *30*(4), 496

Roberts, D. (2010). Vicarious learning: A review of the literature. ***Nurse Education in Practice***, *10*(1), 13-16.

Sachs, S. E. (2020). The algorithm at work? Explanation and repair in the enactment of similarity in art data. ***Information, Communication & Society*****,** *23*(11), 1689-1705. https://doi.org/10.1080/1369118X.2019.1612933

Shaiken, H. (1984). ***Automation and Labor in the Computer Age.*** New York: Holt, Rine-hart, and Winston.

Simon, D. H., & Lieberman, M. B. (2010). Internal and external influences on adoption decisions in multi-unit firms: The moderating effect of experience. ***Strategic Organization***, *8*(2), 132-154.

Swan, J., Bresnen, M., Newell, S., & Robertson, M. (2007). The object of knowledge: The role of objects in biomedical innovation. ***Human Relations*****,** *60*(12), 1809-1837. https://doi.org/10.1177/0018726707084915

Turkle, S. (2008). ***Falling for science: Objects in Mind.*** *MIT Press.*

Vertesi, J. (2020). ***Shaping Science: Organizations, Decisions, and Culture on NASA's Teams***. University of Chicago Press.

Zetka Jr, J. R. (1992). Mass-production automation and work-group solidarity in the post-World War II automobile industry. ***Work and Occupations*****,** *19*(3), 255-271.

Zuboff, S. (1988). ***In the Age of the Smart Machine.*** Basic books.

TABLE 1 Examples of Physical Entities, Transformational Technologies and, Representations

Physical Entity	Transformational Technology	Representation	Citations
Human body	CT scanner	Image produced by scanner	Barley, 1986
Actual car crash	Simulation software	Simulation output	Bailey, Leonardi, & Barley, 2012
Medicine on the shelf	Inventory Software	Screen display of inventory	Barett, Oborn, Orlikowski, & Yates, 2012
Actual motion of satellite	Specific NASA software	Screen displays of satellite motion	Mazmanian, Cohn, & Dourish, 2014
Activists' actions on Wall Street	Twitter algorithm	Activists' Twitter accounts	Dourish, 2016
Actual flight characteristics	Automatic pilot	Flight indicators	Oliver, Calvard, & Potocnik, 2017
Manual review of client records	Algorithm	Recommended documents for review	Kluttz & Mulligan, 2019
Established criteria for classification of art	Algorithm	Recommended classification	Sachs, 2020
Actual field of view of Mars Rover	Software that assembles navigation camera images	Simulated Mars Rover field of view	Vertesi, 2020
Wind movement	Forecasting software	Forecast model	Leonardi, Woo, & Barley, 2021

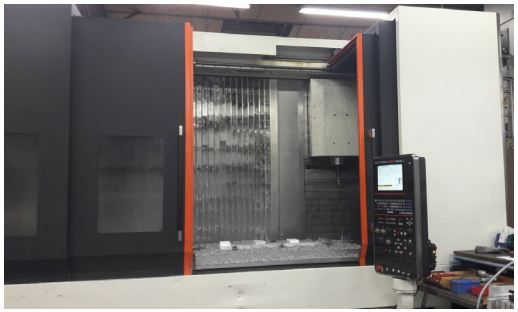
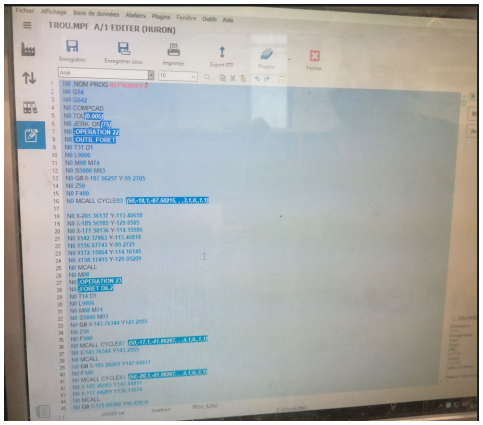
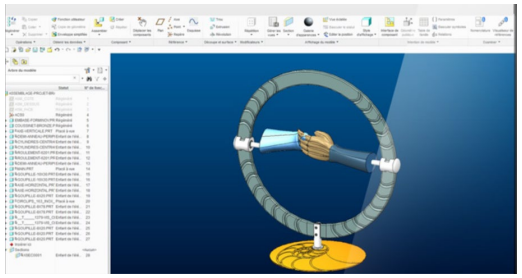
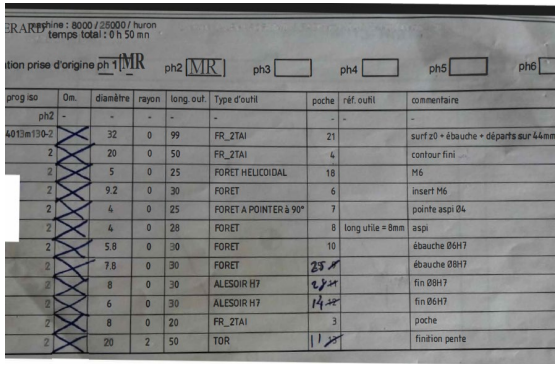
TABLE 2 Work with Digitally Rendered Representations

	Work *within* representations	Work *between* representations and the physical world	Work *from* representations
Role of representations	Create and alter representations from combination of data regarding physical world and knowledge of computational symbols, syntax, and structure	Use representations to buffer professions from physical or used to broker technical infrastructures to support work of others	Representations provide instructions for manipulating or interacting with the physical world
Role of physical world	Distant: Use physical only for validation	At hand: Transform the physical world into representations and take care of the physical world based on representations	Intimate: Work on physical
Role of transformational technology	Program transformational technologies to create and alter representations	Use GUI interface to transform physical world into representations.	None
Capacity to fix misalignment between representations and physical world	Computational thinking and programming	Altering representations in community of practice	Altering physical world in community of practice
Representations experienced as	Computationally Transparent: Pliable and Open	Computationally Semi-Transparent: Pliable and Closed	Computationally Opaque: Unpliable and Closed
Examples of workers	Architects	Engineering technicians	Human Resource Managers
	Engineers	Radiological technicians	Pharmacy Assistants
	Computer Scientists	Computer technicians	Assemblers

TABLE 3 Practical Computational Literacy

Practical Computational Literacy	**Level 1**	**Level 2**	**Level 3**
	Visualize misalignment between the **Representation** and the **Physical Entity.**	Translate between language in the **Transformational Technology** (computational) and the **Representation** (natural).	Edit the symbols and structure of **code in Transformational Technology** to change **Physical Entity**.
Description of skill	Visualize the natural language and assumptions embedded in the representation and identify differences with the physical entity.	Move back and forth between symbols, syntax, structure of code and natural language in the representation to identify correspondence.	Reconceptualize work in transformational technology by modifying symbols or copying, pasting, deleting blocks of code.
Learned by	Listening to others narrating and mimicking the decomposition of representation and visualizing the decomposition of the representation through computational or physical simulations.	Listening to or observing others making these links. Using these experiences to identify patterned links and confirm other links.	Listening to or observing others making these edits. Using these experiences to identify opportunities.
Role of physical entity	Visualize instructions given on representation.	Not used.	Validate correctness of edition made in the code
Role of representation	Identify relationship between natural language and physical entity.	Identify similarities with code in transformational technology.	Not used.
Role of transformational technology	Not used.	Identify deviations vis-à-vis representation.	Solve misalignment between physical and representation by editing it
Used to	Visualize solution to realign the Representation with the Physical Entity.	Understand code in natural language to identify computational solution and provide the basis for editing (next step).	Imagine and assess opportunities to edit the code to realign with the physical and to edit the code in line with these ideas.

TABLE 4 Physical Entity, Transformational Technology and, Representation at Formco

Physical Entity *Machine*	**Transformational Technology** *Coding Software*	**Representation** *Tool Sheet*
	 	

1. For confidentiality reasons, names and initials have been changed, and the name of the operator has been removed.

FIGURE 1 Production Process and Role of Each Worker Group

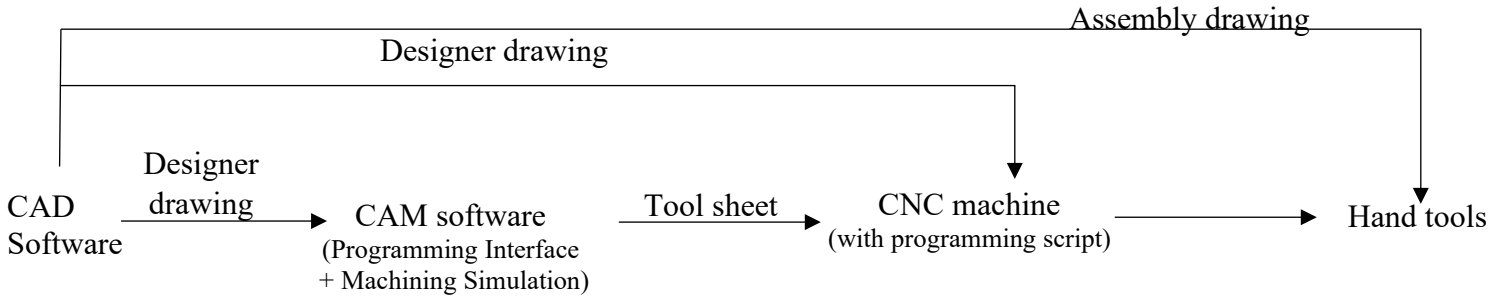
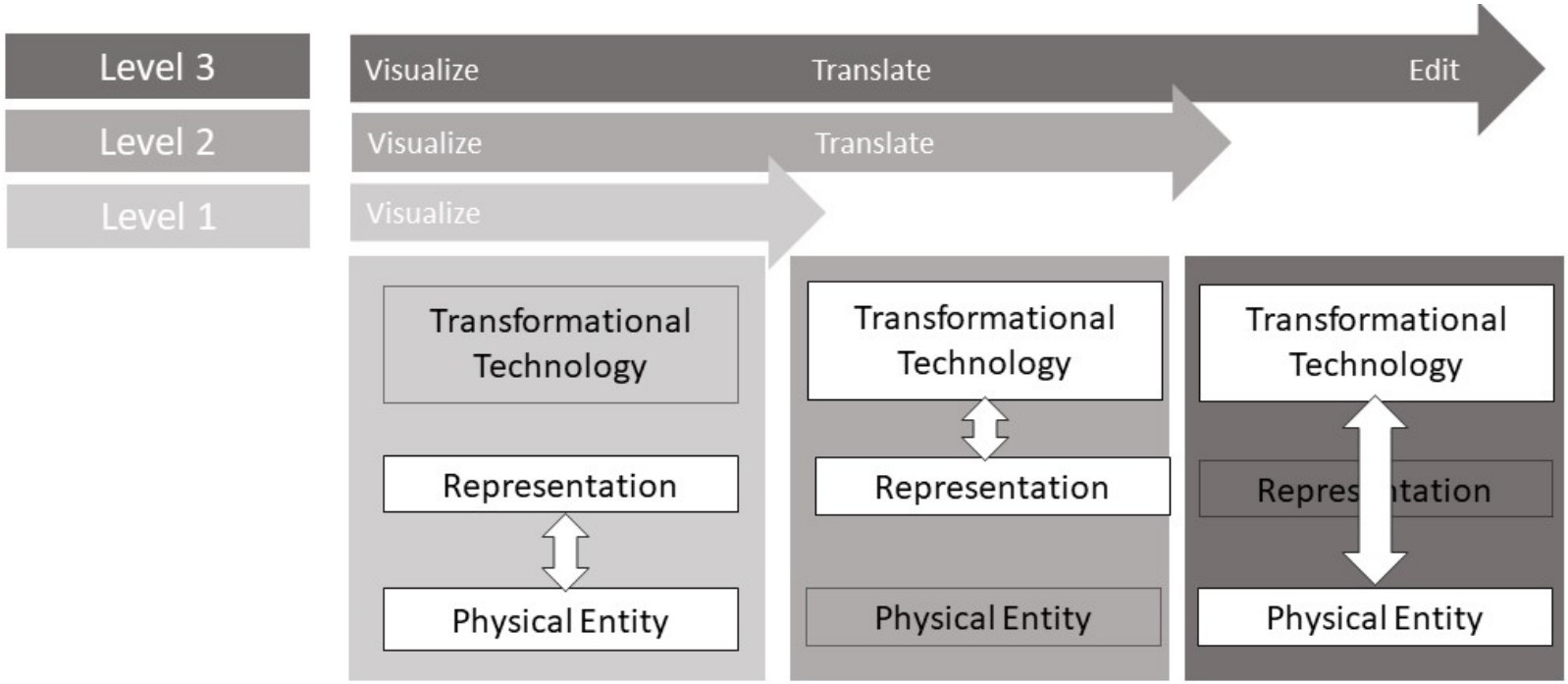
	Designers	Programmers	Machine Operator	Fitters
Work	Design the mold	Program the production instructions	Prepare, launch, and mind the machine	Complete any manual components of mold
Technology used and representations transferred across worker groups				
Locus of practice	Conception	Conception and visualization of the execution	Operation of machine	Manual work with tools
Level of discretion and autonomy	High	High	Low	Low
Work area	Clean open space next to the shop floor	Clean open space next to the shop floor	Dirty and noisy shop floor	Quiet work benches

FIGURE 2 Practical Computational Literacy Skills



New Skill	-Visualize misalignment between the Representation and the Physical Entity	-Translate between language in the Transformational Technology (computational) and the Representation (natural)	-Edit the symbols and structure of code in Transformational Technology to change Physical Entity
Experience Representation as	Closed & Unpliable	Open & Unpliable	Open & Pliable
Solve Problem by	Visualizing solution to realign the representation with the physical entity	Identifying computational solution	Identifying computational solution and editing it

Marjolaine Rostain is a Research Fellow at Warwick Business School. She received her Ph.D from emlyon business school. She is an ethnographer who studies work and technologies.

Ruthanne Huising is an ethnographer of work and organizations. She received her Ph.D. from the Sloan School of Management at M.I.T. and is a Professor of Management and Organizations at Emlyon Business School.